UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO: CIÊNCIA DA COMPUTAÇÃO

ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO EM AMBIENTES CONCORRENTES UTILIZANDO JAVA

Autor 1: Lara Stephanny Lima Gomes.
Autor 2: Ana Karolina Silva Ferreira.

Palavras-chave: Computação Concorrente. Algoritmos de Ordenação. Desempenho. ForkJoinPool. Speedup.

Resumo

A busca por eficiência computacional exige compreender o comportamento dos algoritmos sob diferentes arquiteturas de processamento. Este estudo apresenta uma análise comparativa de quatro algoritmos de ordenação: Mergesort, Quicksort, Counting Sort e Bubble Sort, implementados em Java, contrastando execuções seriais e paralelas. Foram avaliados vetores aleatórios de 10.000, 50.000, 100.000 e 200.000 elementos, com versões paralelas configuradas para 2, 4 e 8 threads e cinco amostras por configuração. Os resultados do CSV mostram ganhos consistentes para Merge Sort e Quick Sort à medida que o tamanho do vetor aumenta e o paralelismo é aplicado. O Counting Sort apresentou tempos muito baixos na forma serial, fazendo com que o overhead de criação e sincronização de threads anulasse possíveis ganhos. Já o Bubble Sort permaneceu significativamente mais lento e pouco escalável, confirmando os limites impostos por sua dependência sequencial e pela Lei de Amdahl.

Introdução

O crescimento do volume de dados tornou relevante a utilização de arquiteturas multicore e de técnicas de computação concorrente (Foster, 1995; Herlihy & Shavit, 2012). Entretanto, a simples adição de threads não garante redução do tempo de execução: o benefício depende da possibilidade de dividir o problema, do balanceamento das tarefas e do custo de comunicação e sincronização. A Lei de Amdahl estabelece que a fração sequencial de uma aplicação limita o speedup máximo alcançável.

Este trabalho investiga o impacto do paralelismo sobre o tempo de resposta de diferentes métodos de ordenação. O MergeSort e o Quicksort representam algoritmos de divisão e conquista; o Counting Sort representa uma abordagem linear não baseada em comparações; e o Bubble Sort funciona como caso de complexidade quadrática e maior dependência sequencial. A análise utiliza as bibliotecas nativas de concorrência da linguagem Java e compara quatro tamanhos de entrada.

Metodologia

Foi desenvolvida uma aplicação de benchmark em Java, com interface gráfica Swing para orquestração e visualização dos testes. A aplicação gera vetores de números inteiros aleatórios com 10.000, 50.000, 100.000 e 200.000 elementos. Cada algoritmo foi executado em uma versão serial e, quando aplicável, em versões paralelas usando pools de 2, 4 e 8 threads.

As implementações paralelas empregam o modelo Fork/Join (Goetz et al., 2006; Oracle), por meio da classe ForkJoinPool (Java Platform, SE) e de tarefas recursivas, permitindo a divisão do trabalho e o balanceamento dinâmico por trabalho roubado (work-stealing). Para cada configuração foram coletadas cinco amostras consecutivas, com medição baseada em System.nanoTime(). Os tempos foram registrados em milissegundos e exportados para o arquivo CSV analisado. O Bubble Sort foi avaliado até 50.000 elementos, pois seu crescimento quadrático produziria tempos excessivos nos maiores vetores.

Resultados e Discussão

Os dados coletados mostram que o efeito do paralelismo depende tanto do algoritmo quanto do tamanho da entrada. Para 200.000 elementos, o Merge Sort apresentou média de 12,2 ms na execução serial, reduzindo para 6,2 ms com 2 threads, 4,4 ms com 4 threads e 3,4 ms com 8 threads. Isso corresponde a uma aceleração aproximada de 3.6 vezes na configuração de 8 threads. O QuickSort também se beneficiou do paralelismo: sua média passou de 8,2 ms no modo serial para 2,0 ms com 8 threads.

Os resultados são coerentes com a estrutura de divisão e conquista desses algoritmos: as sub tarefas podem ser processadas de forma relativamente independente, permitindo melhor utilização dos núcleos disponíveis. O ganho, contudo, não é perfeitamente linear, pois o tempo total inclui criação de tarefas, particionamento, sincronização e combinação dos resultados.

O Counting Sort apresentou comportamento diferente. Para 200.000 elementos, a versão serial registrou média de 0,0 ms, enquanto as configurações paralelas obtiveram 4,0 ms com 2 threads, 2,0 ms com 4 threads e 1,2 ms com 8 threads. Como a execução serial já é muito curta, o overhead do paralelismo supera o benefício potencial. Esse padrão também aparece nos tamanhos menores, nos quais muitos tempos foram registrados como zero milissegundo após o arredondamento.

No Bubble Sort, limitado a 50.000 elementos, a média serial foi de 1259,4 ms. As médias paralelas foram de 1011,8 ms com 2 threads, 1019,4 ms com 4 threads e 1042,6 ms com 8 threads. Embora a configuração paralela reduza o tempo em relação à execução serial, a variação entre 2, 4 e 8 threads é pequena e não apresenta tendência de ganho proporcional. A dependência entre etapas, as barreiras de sincronização e a exclusão mútua limitam a escalabilidade.

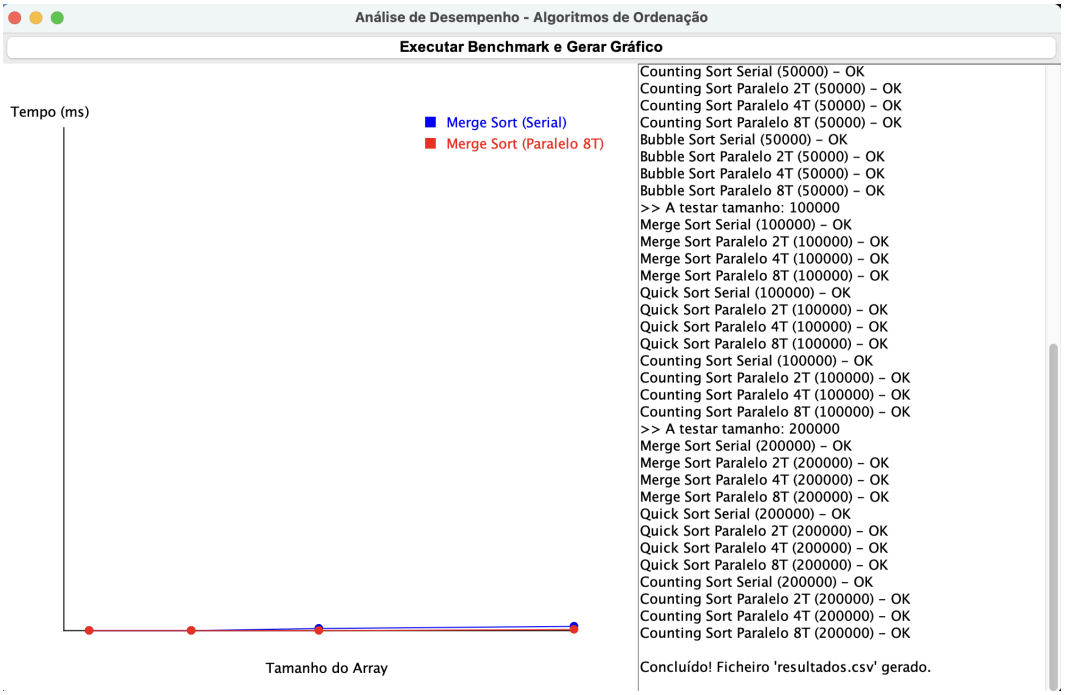


Figura 1 — Interface de benchmark e gráfico de linhas gerado durante os testes.

Conclusão

A análise confirma que o paralelismo é mais vantajoso quando aplicado a algoritmos capazes de dividir o trabalho em subtarefas independentes. Merge Sort e Quick Sort apresentaram reduções relevantes no tempo, especialmente para 100.000 e 200.000 elementos, com destaque para o Merge Sort, que passou de 12,2 ms em média no modo serial para 3,4 ms com 8 threads no maior vetor.

Em contrapartida, o Counting Sort já possui execução serial muito rápida, de modo que a gestão das threads introduz custo adicional. O Bubble Sort apresentou o pior desempenho absoluto e baixa escalabilidade, evidenciando que a paralelização não elimina dependências sequenciais nem os custos de sincronização. Portanto, a escolha de uma estratégia concorrente deve considerar a estrutura do algoritmo, o tamanho da entrada e o equilíbrio entre trabalho útil e overhead.

Referências

FOSTER, Ian. Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Boston: Addison-Wesley, 1995.GOETZ, Brian et al. Java Concurrency in Practice. Upper Saddle River: Addison-Wesley, 2006.

HERLIHY, Maurice; SHAVIT, Nir. The Art of Multiprocessor Programming. Rev. print. Burlington: Morgan Kaufmann, 2012.

JAVA PLATFORM, SE. Class ForkJoinPool. Java API Documentation. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ForkJoinPool.html. Acesso em: out. 2026.

ORACLE. Fork/Join Framework. Java Documentation. Disponível em:

https://docs.oracle.com/javase/tutorial/essential/concurrency/forkjoin.html. Acesso em: out. 2026.

Anexos

O código-fonte da aplicação de benchmark e o arquivo de resultados em formato CSV encontram-se publicamente disponíveis no repositório:

https://github.com/LaraSLGomes/algoritmosDesempenho